SSO Cheat Sheet: Interview Prep (10+ Yrs)

Part 1: Concept, Flow, Logout, Validation, Production

1. One-line answer (bol sakti ho)

> SSO lets a user log in once with a central Identity Provider (IdP) and access many apps without logging in again. Apps don't store passwords; they trust a signed token or assertion from the IdP. It is implemented with SAML 2.0 or OpenID Connect.

Memory trick: Gate-Band-Ride = IdP (gate) - Token (wristband) - App (ride).

2. Core terms

Term	Meaning
IdP	Identity Provider: authenticates user, issues token (Okta, Azure AD, Keycloak, Auth0)
SP	Service Provider: the app that trusts the IdP
SSO	The goal: log in once, access many apps
SAML / OIDC	Protocols to achieve SSO
SCIM	User provisioning (create/update/deactivate). Not authentication
JIT	User created at first login from token claims

SSO = goal, SAML/OIDC = protocols. (Delhi pahunchna = goal, train/flight = raaste.)

3. Flow: R-L-T-V-S

1. **R**edirect: app sends user to IdP
2. **L**ogin: IdP authenticates (+ MFA)
3. **T**oken: IdP issues signed assertion / ID token
4. **V**erify: app validates signature and claims

5. **S**ession: app creates its own local session

Second app: IdP session already exists, so no login page. That is SSO.

4. Architecture

- **Frontend (React/Angular):** redirects to IdP, handles callback
- **Backend (Node/Spring):** validates token, maps user in DB, creates session
- **IdP:** Okta / Azure AD / Keycloak
- **SCIM:** IdP pushes user create/disable events to app
- **DB:** user to IdP-subject mapping, roles, per-tenant IdP config

5. Two sessions (most asked)

- **IdP session** lives at the IdP. **App session** lives in the app. Independent lifetimes.
- App verifies token only at login, then does not ask the IdP again.
- So IdP logout does not auto-logout apps.

Memory trick: Do Ghar, Do Chaabi.

Interview answer: cause (two sessions) + fix (Single Logout / short session) + trade-off (SLO unreliable, short expiry costs UX).

6. Logout types

	Front-channel	Back-channel
Path	IdP to browser to app (iframes/redirects)	IdP server to app server (logout token)
Reliability	Low (tab closed, 3rd-party cookies blocked)	High
App needs	Logout URL	Session store lookup by `sid` / `sub`

- Back-channel: app validates logout token, finds session by `sid`, **deletes it server-side**.
- Stateless JWT app: short-lived tokens (5-15 min), denylist of revoked `sid` / `jti` in Redis, short refresh cycle.

- SLO is hard: many apps, some down, some unsupported, partial logout. Many teams accept short sessions instead.

7. Token validation: S-I-A-E-N

Letter	Check
S	**Signature** (IdP public key, from JWKS/metadata)
I	**Issuer** (`iss`) matches expected IdP
A	**Audience** (`aud`) is MY app
E	**Expiry** (`exp` , `nbf`)
N	**Nonce** (OIDC) / `InResponseTo` (SAML)

SAML extras: `Recipient` , `NotOnOrAfter` , replay protection.

Common mistake: skipping `aud` . A token made for another app could then work in yours.

Key rotation: fetch keys from JWKS, cache them, re-fetch once on unknown `kid` . Never hardcode the certificate.

8. SP-initiated vs IdP-initiated

- **SP-initiated:** user starts at app. App can bind `state` / `InResponseTo` , so CSRF and replay are easier to prevent. **Preferred.**
- **IdP-initiated:** user clicks tile at IdP portal. App never sent a request, so it cannot verify the response belongs to one. Security teams avoid it.

9. Provisioning

- **JIT:** creates user at first login. Cannot remove users who never log in again.
- **SCIM:** IdP pushes create / update / **deactivate**. Needed for proper offboarding.

10. Production and architecture

- **IdP down:** existing app sessions keep working until expiry; new logins fail. Mitigate with sensible session lifetime, clear error page, break-glass admin login, IdP health monitoring.
- **Multi-tenant B2B SaaS:** store per-tenant IdP config (protocol, issuer, metadata/JWKS URL, client ID, certs, attribute mapping). Detect tenant by email domain or subdomain. **Validate token `iss` matches that tenant's configured issuer.**
- **Authorization:** IdP authenticates; app usually authorizes by mapping IdP group claims to roles.
- **Frontend token storage:** prefer HttpOnly + Secure + SameSite cookie set by backend. localStorage is exposed to XSS. SPA doing the flow itself: authorization code + PKCE, tokens in memory.

11. Tricky one-liners

Question	Answer
SSO means one password for all apps?	No. Apps never see passwords; only the IdP does.
Single point of failure?	Risk is centralized, so MFA, conditional access and monitoring are a must. Net security usually improves.
Do SAML apps use passwords?	No, they trust the signed assertion.
SSO = SAML?	No. SSO is the goal, SAML is one protocol.

12. Common mistakes

- Saying "SSO = SAML"
- Saying "OAuth is for login" (OAuth = authorization; OIDC adds identity)
- Only explaining the happy flow; seniors are asked about logout, expiry, validation, IdP failure
- Forgetting `aud` and issuer-to-tenant binding
- Long, vague answers. Format: **short answer first, then detail**

13. Verified company reports

Mastercard, Senior Software Engineer (Java), Pune, Feb 2026 (Glassdoor candidate report)

- Round 1: SSO, SAML, OIDC, OAuth 2.0
- Round 3 (Technical SME): how SSO works, SCIM in SSO, Spring Boot Security, AWS Lambda, SAML vs OIDC vs OAuth 2.0
- Feedback: be more direct and concise

Sources: Glassdoor Mastercard SSE interview pages (glassdoor.com.au, glassdoor.fr).

Commonly expected (blog, not company-reported): OIDC auth-code flow with PKCE; where to store tokens in an SPA; enterprise SSO for a B2B product; threat model of localStorage tokens. Source: techinterview.org, Frontend Authentication 2026.

14. Priority questions

- **P1:** What is SSO? Explain the flow. IdP vs SP. SSO vs SAML.
- **P2:** SP vs IdP-initiated. SSO session vs app session. Why is Single Logout hard? Front vs back-channel logout.
- **P3:** Multi-tenant SSO design. Full token validation. IdP outage handling. JIT vs SCIM. Key rotation.

15. Top 10 quick Q&A

1. **What is SSO?** Log in once at the IdP, access many apps.
2. **IdP vs SP?** IdP proves who you are; SP decides what you can do.
3. **Why two sessions?** App verifies the token once and keeps its own session.
4. **Front vs back-channel logout?** Browser-based vs server-to-server; back-channel is more reliable.
5. **What to validate in a token?** Signature, issuer, audience, expiry, nonce.
6. **Why check `aud`?** Stops a token meant for another app being accepted by yours.
7. **JIT vs SCIM?** JIT creates on login; SCIM also deactivates.
8. **IdP down?** Existing sessions continue; new logins fail.
9. **Multi-tenant?** Per-tenant IdP config; `iss` must match tenant.
10. **SSO vs SAML?** Goal vs protocol.

16. Checklist: explain without help

- [ ] SSO in one line, with the Gate-Band-Ride example
- [ ] R-L-T-V-S flow on a whiteboard
- [ ] Why IdP logout does not log out apps
- [ ] Front-channel vs back-channel logout
- [ ] S-I-A-E-N validation and why `aud` matters
- [ ] SP-initiated vs IdP-initiated
- [ ] JIT vs SCIM
- [ ] Multi-tenant SSO design with issuer binding
- [ ] IdP outage behavior
- [ ] Token storage in SPA

Next: Part 2

SAML vs OAuth 2.0 vs OIDC (deep dive), PKCE, refresh tokens, Spring Boot / Node implementation.